



PROJECT REPORT

Cat Overlord

TEAM	error404
REFERENCE ID	7P91DRIN
THEME	For Developers — #hackthekitty 2026
TYPE	Electron desktop AI companion

Project Name: Cat Overlord **Team Name:** error404 **Reference ID:** 7P91DRIN **Theme:** For Developers
#hackthekitty 2026

1. Executive Summary

Cat Overlord is a personality driven desktop AI assistant, shaped like your cat, that helps developers manage the small friction points of their workflow checking calendars, drafting emails, summarizing inboxes, controlling music, defining words, opening apps, and analyzing what's on screen all through natural conversation with an in character sarcastic feline overlord. Built as an Electron desktop pet powered by Claude Sonnet 4.5, it makes routine developer admin tasks faster and considerably more entertaining.

2. Project Overview

2a. Why we're building what we're building

Developers lose small, cumulative chunks of time throughout their day switching contexts checking calendars in another tab, opening Gmail to draft a quick reply, hunting for the app that needs to launch, remembering to take a Pomodoro break. Individually these are trivial; cumulatively they cost focus. Most existing assistants either sit in a chat window (Copilot, ChatGPT) or run as headless voice assistants (Alexa, Cortana), neither of which feels present on the developer's desktop the way a real coworker or pet would.

Cat Overlord addresses this gap with a floating, always on top desktop cat that shares personality traits derived from an uploaded photo, learns about its user over time, and executes multistep tasks autonomously through natural language. It

removes the "open another app to do X" friction while adding a genuinely fun character to the developer's environment.

2b. How it relates to the theme

Cat Overlord targets the "For Developers" track: "cat themed dev tools... or anything that makes coding more feline friendly." It is exactly that a cat themed development companion that:

- Sits on the developer's desktop while they code, floating and interactive
- Handles developer adjacent admin tasks: calendar, email, notetaking (via summaries), reminders, focus timers, opening dev tools
- Analyzes what's on screen including code and can suggest fixes or explain what the developer is looking at
- Adds personality and delight to the coding environment through mood driven animations, TTS speech, and situational sarcasm

It is not a general purpose assistant that happens to be cat shaped; it is a cat shaped assistant designed for the specific workflow of someone who spends their day in an editor.

2c. Target Audience

Primarily developers and computer science students who want a personable, low friction way to handle the interstitial tasks around coding checking calendar, drafting emails, playing music, defining terms without breaking flow. Secondly, anyone who works knowledge intensive jobs at a computer and would benefit from an incharacter AI companion.

3. Key Features

Cat Overlord bundles the following capabilities into a single always on top Electron desktop pet:

Core AI Agent

- **Personality generation from photo** Claude Vision analyzes the uploaded cat photo and generates a unique personality that colors every reply
- **Persistent memory** the cat learns durable facts about its user (name, contacts, writing style, preferences) via a reflection step after each interaction and applies them to future tasks
- **Multistep autonomous tasks** a single natural language command can chain multiple tools (e.g. "handle my 3 PM meeting" → read calendar → read attendee emails → draft followup)
- **Emotional mood system** the cat self reports a mood from its emotional response, driving animations and overlay effects (happy, angry, confused, sleepy, drafting, email, checking calendar, etc.)
- **Personalized launch greeting** uses stored memory to greet the user by name with context awareneass

Developer Tools

- **Screen analysis** silently captures a screenshot on request and describes what the developer is looking at, including code (can also suggest fixes)
- **Application launcher** opens installed Windows apps by fuzzy name search with a hardcoded denylist blocking system shells and admin tools
- **Focus/Pomodoro timer** with countdown UI and TTS break announcements
- **CPU/RAM monitoring** with a fire overlay animation when system resources spike
- **Dictionary lookup** for word definitions

Productivity Tools (Google + Media Integrations)

- **Google Calendar** reads events across all subscribed calendars (primary + secondary), creates reminders
- **Gmail** reads and summarizes emails (with body extraction), drafts messages (never sends always leaves as drafts for user review)
- **Detailed summaries to file** long summaries are written to `.txt` files stored in a dedicated app managed folder (`%LOCALAPPDATA%\vcatsummaries`), accessible through an in app panel
- **Spotify** plays songs on demand via the Spotify Web API on the user's Premium account
- **Weather** current conditions via IP geolocation + OpenMeteo
- **News** latest world headlines via BBC RSS with in character commentary

UserFacing Personality Layer

- **Text to speech** with mood aware voice modulation (rate/pitch/volume vary per mood) and prepended cat sound effects
- **Mood driven animations and overlays** hearts on happy, anger symbol on angry, floating musical notes when a song plays, question marks when confused, Z's when sleepy, and more
- **Speech bubble** with length adaptive display time, hover pause, and overflow scrolling
- **Voice character selection** (male / female / neutral) chosen at cat upload
- **System tray control center** recall cat to center, hide/show, quit
- **Session chat history panel** and **memory transparency panel** user can inspect and delete what the cat knows about them

Persistence and Lifecycle

- **Cat persists across launches** cutout image saved to user Data, personality and memory reloaded on restart
- **Delete cat flow** dedicated confirmation dialog with an optional "also delete my saved summaries" checkbox

🐾 4. Technology Stack

Layer	Technology
Desktop shell	Electron (transparent, frameless, always on top window)
Frontend	HTML / CSS / vanilla JavaScript
Backend	Python 3.11+ with FastAPI (uvicorn ASGI server)
AI model	Anthropic Claude Sonnet 4.5 (tool use loop for agent reasoning; Claude Vision for photo personality generation and screen analysis)
Text to speech	Microsoft Edge TTS (backend generated MP3 audio)
Background removal (frontend)	@imgly/backgroundremoval in browser library
Calendar and mail	Google Calendar API v3, Gmail API v1 (OAuth 2.0 Authorization Code flow)
Music	Spotify Web API via spotipy (Authorization Code flow, Premium account required for playback)
Weather	OpenMeteo API + IPbased geolocation via ipapi.com
News	BBC News World RSS feed via feed parser

Layer	Technology
Dictionary	Free Dictionary API
App discovery	Windows PowerShell Get Start Apps (canonical Start Menu app list)
System monitoring	psutil for CPU/RAM sampling
Image handling	Pillow, pyautogui (screenshots)
HTTP client	Requests

5. Technical Architecture

Cat Overlord is a two process desktop application. The Electron frontend and Python backend communicate exclusively over HTTP on `localhost:8000` a strict boundary that let two developers work independently in parallel throughout the build.

High Level Diagram



Request Flow (MultiStep Command)

The multistep agent flow is the product's core capability. Example: user asks "handle my meeting today with the Statistics TAs."

1. Frontend sends `POST /command {"text": "handle my meeting..."}` to backend.

2. Backend loads current `personality.json` + `memory.json` + IST time and injects into system prompt.
 3. Claude receives the prompt and decides to call `read_calendar` first → tool returns today's events across all Google calendars.
 4. Claude sees the Stats1 TA session, decides to call `read_emails` with a matching query and `include_body=True` → tool returns relevant email bodies.
 5. Claude synthesizes context and calls `draft_email` → Gmail draft is created.
 6. Claude produces a short, in character reply with a `MOOD:` tag.
 7. Backend parses the mood, invokes Edge TTS to render an MP3, runs the reflect step (extract new durable facts → append to `memory.json`), and returns `{reply, mood, tools_used}` to the frontend.
 8. Frontend displays the reply in the speech bubble, plays the TTS audio, and triggers the mood animation.
-

🐾 6. Security Considerations

Security was designed in from the start, given the tool has read/write access to real user data (Gmail, Calendar), can play audio, can launch applications, and can capture screenshots.

Design decisions and safeguards

- **Least privilege OAuth scopes.** Google OAuth requests only the scopes actually needed `gmail.readonly`, `gmail.compose`, `calendar` and no more. The cat drafts emails but never sends them; the `gmail.send` scope is deliberately not requested. Similarly, Spotify OAuth requests only `usermodifyplaybackstate` and `userreadplaybackstate`.
- **Secrets in `.env`, gitignored.** All API keys and OAuth secrets (`ANTHROPIC_API_KEY`, `SPOTIFY_CLIENT_ID`, `SPOTIFY_CLIENT_SECRET`) live in `backend/.env` which is gitignored. OAuth token caches (`token.json`, `spotify_token.json`) are also gitignored. No secret ever enters version control.
- **Applicationlauncher guardrails.** The `open_application` tool is constrained by both an allowlist and a denylist:
- *Allowlist:* only apps returned by Windows' `GetStartApps` (userfacing Start Menu apps) can be considered as launch candidates. Arbitrary executables on disk, PATH entries, and Program Files scanning are all excluded.
- *Denylist:* a hardcoded list of system shells and administrative tools (`cmd`, `powershell`, `pwsh`, `wt`, `regedit`, `services`, `gpedit`, `diskmgmt`, `devmgmt`, `eventvwr`, `mmc`, `msconfig`, `compmgmt`, `secpol`, `perfmon`, `taskschd`, `taskmgr`, `netsh`, `cmdkey`, `psexec`) is refused even if they appear in `GetStartApps`.
- *Wordboundary matching* prevents false positives (e.g. `wt` matching inside Newton).
- *Fuzzy "did you mean" fallback:* requests that don't match the allowlist return suggestions rather than opening a random executable.
- **Email is draftonly, never send.** Even though the cat can compose emails, the `draft_email` tool creates Google drafts and stops. All sending requires the user to open Gmail and click "Send" themselves, so the AI cannot autonomously send anything on the user's behalf.
- **Summary file deletion is scoped.** The `delete_summary_file` helper verifies via `os.path.commonpath` that the target file lies inside `%LOCALAPPDATA%\vcats\summaries` before deleting. It refuses to delete anything outside that folder even if `summaries.json` were tampered with to reference a system file, the deletion would be refused.
- **Photo validation.** Uploaded cat photos are checked by Claude Vision (`is_cat: true/false`) before personality generation and cutout storage. Noncat images are rejected with a sassy incharacter error, preventing arbitrary photos from becoming the desktop overlay.

- **Cat image validation** is lenient toward cats, strict against noncats kittens, cartoons, and odd poses are accepted; people, cars, landscapes are rejected. Defaults to accepting on parse failure so real cats aren't blocked by a JSON hiccup.
- **Screen analysis is opt-in.** The screenshot tool only captures when the user explicitly asks the cat to look at their screen ("what's on my screen"). No silent or scheduled captures. Screenshots are processed via Claude Vision and not persisted to disk after analysis.
- **No arbitrary code execution.** The cat cannot run shell commands, evaluate arbitrary code, or install anything. Every "action" the cat takes goes through one of the explicit, handwritten tools in `tools.py`.
- **Local only backend by default.** FastAPI binds to `127.0.0.1:8000` the backend is not exposed to the network. All communication is between the Electron frontend and Python backend on the same machine.
- **Memory and personality are user owned.** The user can inspect what the cat has learned about them via the memory panel, delete individual facts, or wipe everything with "forget everything." Nothing about the user is sent to third party services except what's necessary for a specific tool call (e.g. calendar events sent to Claude for reasoning about them).
- **Google refresh token expiry acknowledged as an operational risk.** In OAuth "Testing" mode, Google expires refresh tokens after 7 days. This is documented and the setup requires periodic reauth; a production version would move the OAuth app to "Published" status.

7. Testing Matrix

Manual test cases run to verify the project works as intended. Cat Overlord was tested continuously across development sessions; the table below consolidates the key end-to-end verification cases.

Feature / Flow	Steps	Expected Result	Actual Result	Pass/Fail
App launch (fresh)	Start backend and frontend with no saved cat	Upload screen appears; no cat visible	Upload screen appeared	Pass
App launch (with saved cat)	Restart app after saving a cat	Saved cat loads, no upload screen, personalized greeting speaks	Cat loaded and greeted with saved memory context	Pass
Cat upload real cat photo	Upload a cat photo	Background removed; personality generated; cat appears as floating cutout	Cutout appeared, personality set	Pass
Cat upload noncat image	Upload a landscape or person	Rejection popup with in character message; upload screen stays	Rejection popup displayed the sassy message	Pass
Cat persistence	Quit via tray, relaunch	Same cat and personality; memory intact	Persisted correctly	Pass
Google Calendar read	Ask "what's on my calendar today?" (with events across multiple calendars)	All events from primary + secondary calendars listed	Initially returned <code>[]</code> due to primary only query; fixed to iterate all calendars via <code>calendarList.list()</code>	Pass (after fix)

Feature / Flow	Steps	Expected Result	Actual Result	Pass/Fail
Gmail read/summarize	Ask "summarize my last 5 emails"	.txt file appears in %LOCALAPPDATA%\vcats\summaries with detailed multiemail summary; brief in character reply pointing to the file	File appeared; summary was detailed and clean	Pass
Gmail draft	Ask "draft an email to Prof. Rao asking to reschedule"	Draft created in Gmail Drafts folder; never sent	Draft appeared in Gmail; not sent	Pass
Multistep agent	Single command combining calendar + email + draft (e.g. "handle my TA session today")	Tool calls execute in sequence; final draft matches context from calendar + related emails	Chained correctly across tools	Pass
Reminder creation	Ask "set a reminder for tomorrow at 3 PM"	Calendar event created at correct time	Event created	Pass
Spotify playback direct song	Ask "play Paper Rings by Taylor Swift" (Spotify open + Premium account)	Track begins playing on active device; music note overlay animates	Played correctly with field filtered search (track: + artist:) after loose search was returning wrong tracks	Pass (after search precision fix)
Weather	Ask "what's the weather?"	In character reply with real temperature and condition for user's city	Returned 32°C overcast Chennai; correct	Pass
News	Ask "what's in the news?"	23 recent BBC headlines with in character commentary	Returned real headlines with dry cat commentary	Pass
Screenshot analysis	Open code with a bug, ask "what's on my screen?"	Cat describes the screen content accurately; can identify issues in visible code	Described correctly	Pass
App launcher legit app	Ask "open notepad"	Notepad launches	Opened	Pass
App launcher denylist	Ask "open cmd" or "open regedit"	Refused with system tool message; nothing opens	Refused correctly	Pass
App launcher typo tolerance	Ask "open chorme"	Agent autocorrects to Chrome and opens; or if it passes the typo through, "did you mean Chrome?" suggestion returned	Agent autocorrected in context; direct tool call gives suggestion	Pass
App launcher unknown app	Ask "open blender" (not installed)	"Not installed" error with optional suggestions; nothing opens	Behaved correctly	Pass

Feature / Flow	Steps	Expected Result	Actual Result	Pass/Fail
Focus timer	Ask "start a 5minute timer"	Countdown UI appears; TTS break announcement at zero	Timer ran and announced	Pass
Word definition	Ask "define serendipity"	Clean dictionary definition returned	Definition returned	Pass
System resource fire overlay	Trigger high CPU (compilation or video)	Fire overlay appears near cat while CPU sustained high; clears when it drops	Fire appeared and cleared correctly	Pass
Memory learning + application	Give preference ("sign my emails as Soka"), then ask for another draft without repeating it	Second draft applies the learned preference automatically	Draft came out signed correctly without being told twice	Pass
Memory panel delete fact	Delete a fact from panel, request an action that would use it	Cat no longer applies deleted fact	Behavior updated after deletion	Pass
Chat history panel	Open panel after several messages	Last 10 message pairs displayed; empty on relaunch (session only)	Displayed correctly; cleared on relaunch	Pass
Summaries panel open file	Click a summary row	<code>.txt</code> opens in default editor via <code>shell.openPath</code>	Opened correctly	Pass
Summaries panel delete entry	Click X on an entry (with confirm)	Entry removed AND <code>.txt</code> file removed from AppData; safety check prevents deletion outside the summaries folder	Entry and file removed; safety check verified	Pass
Speech bubble long reply	Ask for a deliberately long reply	Bubble scales or scrolls; text never cut off	Handled correctly	Pass
Speech bubble hover pause	Hover over bubble	Autohide pauses; resumes on mouse leave	Pause/resume working	Pass
Tray recall cat	Move cat offscreen; click "Center cat"	Cat returns to screen center on a single click	Worked in one click after removing stale <code>isVisible()</code> guard	Pass (after fix)
Tray hide/show	Toggle "Hide cat" and "Show cat"	Window hides and reappears without quitting app	Worked correctly	Pass
Tray quit	Click "Quit"	Application closes cleanly; no zombie process	Closed cleanly	Pass
Cat deletion keep summaries	Click "Delete cat" with	Cat removed; summaries panel + files still present after new upload	Preserved correctly	Pass

Feature / Flow	Steps	Expected Result	Actual Result	Pass/Fail
	checkbox unchecked			
Cat deletion wipe summaries	Click "Delete cat" with checkbox checked	Cat removed; summaries panel empty; AppData folder empty	Wiped correctly	Pass
Mood animations	Compliment cat, then insult it	Hearts fire for happy; red tint + anger symbol for angry	Both animations fired reliably after adding angry mood and refactoring <code>_pick_mood</code> to accept an agent reported mood tag	Pass (after fix)
TTS normal reply	Send any command	Reply spoken with cat's chosen voice, cat sound prepended	Working correctly	Pass
TTS mute	Toggle mute; send another command	Reply displayed in bubble, no audio plays	Working correctly	Pass

🐾 8. Future Improvements

Given more time, we would extend the project in these directions:

- Move the Google OAuth app from Testing to Published so the 7day refreshtoken expiry stops being an operational concern
- Calendar write operations (reschedule, cancel) with a twostep confirmation flow so the destructive actions are user approved before execution
- **File organization and clutter cleanup** let the cat organize files and tidy up the developer's laptop (sorting downloads, grouping loose files into folders, clearing desktop clutter) on request, with a previewandconfirm step before anything is moved or deleted
- Multimonitor support for the cat's spawn position, offscreen recovery, and screenshot capture (currently primary display only)
- Cross-platform build currently Windowsonly due to `GetStartApps` , `%LOCALAPPDATA%` , and `os.startfile` . macOS and Linux ports would need platform specific launchers.
- Streaming Claude responses so the speech bubble fills in as the reply is generated, rather than waiting for the full tooluse loop to finish
- Local model fallback so basic conversational functions work without an internet connection or API credits
- Voice input (STT) so the developer can talk to the cat, not just type pairs naturally with the existing TTS
- Additional integrations GitHub (PR reviews, issue summaries), Notion, Linear extending the developer workflow surface area
- Sandboxed screen analysis so the screenshot never leaves the machine (locally run vision model instead of Claude Vision) for privacy conscious developers

🐾 9. Tools Used

- **Claude Code (Anthropic)** primary AI pair programming assistant for backend development, refactoring, and debugging
 - **Cursor** AI assisted code editor for frontend development
 - **VS Code** primary editor for code review, git operations, and terminal work
 - **GitHub** source control and collaboration between the two team members
 - **Xbow Screen Recorder** screen recording for capturing test runs and producing the demo video
 - **Anthropic Claude** the model powering the desktop assistant itself at runtime (Claude Sonnet 4.5 for reasoning and Claude Vision for photo/screen analysis)
 - **Google Cloud Console** for OAuth 2.0 credential setup for Gmail and Calendar
 - **Spotify Developer Dashboard** for OAuth app registration and API credentials
-

🐾 10. Learnings & Takeaways

Several concrete lessons emerged from building Cat Overlord:

- **A hard frontend/backend boundary paid off.** Because the Electron app and Python backend communicated exclusively over HTTP on `localhost:8000`, we could work in parallel most of the time. The times we lost hours were the times either developer's AI reached across the boundary and edited the other's files a discipline problem, not a design problem.
 - **Persistent memory + a reflection step is a small change with a large behavioral impact.** Adding a single extra Claude call after each command to extract durable facts made the assistant feel dramatically more "yours" than a stateless bot. The UX difference is bigger than the engineering cost.
 - **The mood system had to be model reported, not tool inferred.** Our first pass computed mood from which tools ran, which meant conversational replies (no tools) always had the same mood. Letting Claude self report an emotional mood in a tag on the final line of its reply gave the animation system real variety with almost no extra code.
 - **Windows OneDrive Desktop redirection is a real gotcha.** Our summaries feature "worked" by returning success but files landed in a Desktop path the user couldn't see. We later moved summaries to a dedicated `%LOCALAPPDATA%\vcats\summaries` folder so the app owns the file lifecycle endtoend.
 - **Guardrails should be structural, not enumerated.** Rather than trying to enumerate every "bad" app the launcher should refuse, the working design was: allowlist from `GetStartApps`, denylist for known shells and admin tools, wordboundary regex to avoid false positives. Categorical rules beat manuallymaintained lists.
 - **Detailed summaries require full email bodies, not snippets.** Gmail's snippets are ~100 characters enough for triage, not for summarizing. Adding an `include_body=True` flag to `read_emails` unlocked the summary feature. Making it optin kept normal reads cheap.
 - **OAuth setup is the most fragile part of a working demo.** Google's 7day refresh token expiry in Testing mode and Spotify's strict redirect URI matching (with recent moves toward requiring `127.0.0.1` over `localhost` for HTTP) both cost us hours we hadn't budgeted for.
-

🐾 11. Acknowledgments

- **Anthropic** for the Claude API (Claude Sonnet 4.5 and Claude Vision) the model powering the entire assistant
 - **Google** for the Gmail and Calendar APIs
 - **Spotify** for the Web API and their developer platform
 - **OpenMeteo** for free, nokey weather data
 - **BBC News** for a public RSS feed we could rely on
 - **The Free Dictionary API** for word definitions
 - **@imgly/backgroundremoval** for excellent in browser background removal
 - **spotipy, feedparser, psutil, pyautogui, Pillow** the Python libraries we leaned on heavily
 - **The organizers of #hackthekitty 2026** for a theme that made building this deeply fun
-

🐾 Submission Checklist

- ☒ Video demo (HD, 720p+)
 - ☒ README.md (prerequisites, run instructions, configuration)
 - ☒ Project report (this document in `documentation/`)
 - ☒ Source code in `src/`
 - ☒ No unrelated files, executables, autogenerated code, or package folders (`node_modules/` , `venv/` , `build/` , `dist/` , etc.)
-

End of Report